Bài 2. Intelligent Agents

Mục tiêu

- Khái niệm agent và environment trong AI.
- Phân biệt các loại môi trường và ảnh hưởng đến thiết kế agent.
- Hiểu khái niệm rationality và cấu trúc của agent.
- Liên hệ tới các ứng dụng thực tế: chatbot, computer vision, LLM, robotics.
- Trải nghiệm mô phỏng grid-world agent qua file Python notebook.

1. Giới thiệu

Gemini

Có phải là agent không?

1. Giới thiệu

- *Agent là bất cứ thực thể nào có thể được xem là cảm nhận môi trường qua sensors và tác động lên môi trường qua actuators.*

- **Environment:** thế giới mà agent tương tác.
- **Sensors:** thu nhận thông tin từ môi trường.
- **Percepts:** dữ liệu cảm nhận (ảnh, âm thanh, văn bản...).
- **Actuators:** công cụ thực thi hành động.
- **Actions:** tác động lên môi trường (di chuyển, trả lời, điều khiển…).

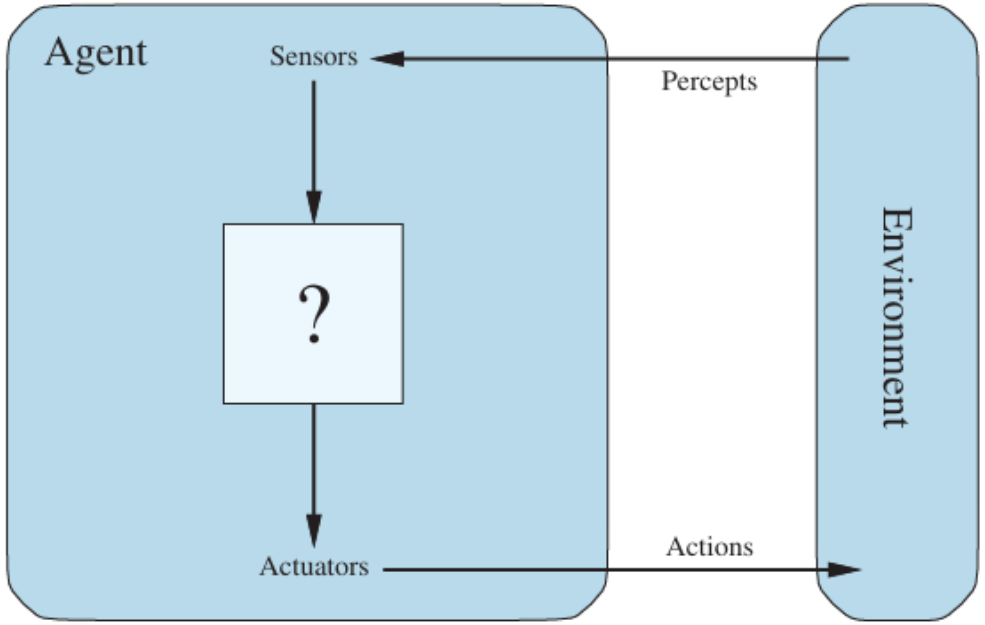


Figure 2.1 Agents interact with environments through sensors and actuators.

2. PEAS: mô tả bài toán của agent

Khi thiết kế agent, phải đặc tả **Task Environment** bằng khuôn **PEAS**: **Performance measure (**tiêu chí để đánh giá agent có hoạt động tốt hay không**)**, **Environment**, **Actuators**, **Sensors**

Lưu ý: Mục tiêu có xung đột → cần **trade-offs** (an toàn, nhanh, thoải mái, chi phí…).

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Figure 2.4 PEAS description of the task environment for an automated taxi driver.

2. PEAS

Tạo bảng PEAS cho Gemini/ChatGPT?

PEAS: Gemini/ChatGPT

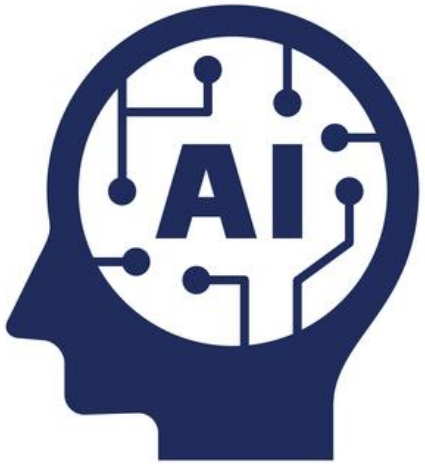
Agent	**Performance (P)**	**Environment (E)**	**Actuators (A)**	**Sensors (S)**
Gemini **ChatGPT**	• Nội dung chính xác, mạch lạc • Tuân thủ hướng dẫn • Hữu ích	• Phiên hội thoại • Công cụ tích hợp (nếu có) • Tài liệu do người dùng cung cấp	• Sinh văn bản • (Có thể gọi công cụ qua API)	• Văn bản người dùng • File/tài liệu đính kèm

Các hệ **LLM agentic** hiện đại thường bổ sung **tool use** (gọi API, duyệt web), **memory**, **planning** (lập kế hoạch đa bước), biến chúng thành **goal-based/utility-based/learning agents** trong thực hành.

3. Rational Agent là gì?

Agent hợp lý (rational agent) là agent chọn hành động nhằm tối đa hóa **expected utility** dựa trên:

- Chuỗi percept đã nhận
- Kiến thức có sẵn
- Mục tiêu (performance measure)



Good Behavior: Rationality

3. Rationality ≠ Omniscience

- Omniscience (toàn tri): biết mọi thứ, kể cả tương lai.
- Rationality: chỉ cần chọn hành động tốt nhất dựa trên thông tin hiện có.
- Ví dụ:Xe tự lái không biết chắc xe trước sẽ phanh gấp → chọn hành động hợp lý dựa vào xác suất.
- ChatGPT không “biết hết”, nhưng dựa vào prompt + training data để trả lời hợp lý nhất.

3. Limited Rationality

- Vấn đề: tính toán hành động tối ưu có thể tốn kém tài nguyên (thời gian, bộ nhớ).
- Limited rationality: agent phải hành động kịp thời, không thể đợi tính toán xong toàn bộ.
- Giải pháp:Approximation (xấp xỉ)
- Heuristics (chiến lược tìm kiếm nhanh)
- Learning (học từ kinh nghiệm để giảm chi phí tính toán)
- Ví dụ:AI trong game chọn nước đi tốt đủ nhanh, không phải tối ưu tuyệt đối. Robot hút bụi Roomba chỉ cần dọn sạch “tốt đủ”, không cần tối ưu quãng đường toàn cục.

3. Limited Rationality: ví dụ thực tế

- LLM (ChatGPT) như rational agent:Không toàn tri, chỉ sinh phản hồi dựa trên dữ liệu đã học + input hiện tại.
- Có tính “limited rationality” vì không tính toán vô hạn → dùng heuristic sinh văn bản.
- Robotics: robot không thể xét mọi kịch bản, phải chọn hành động hợp lý trong giới hạn.

3. Limited Rationality: Thảo luận

- Một chatbot “hợp lý” cần gì để trả lời đúng?
- Ví dụ về **Limited Rationality**?

3. Limited Rationality: Thảo luận

- **Câu hỏi 1: Một chatbot "hợp lý" cần gì để trả lời đúng?**
- Hiểu đúng input (ngữ nghĩa, ngữ cảnh)
- Có tri thức nền (kiến thức học được / dữ liệu huấn luyện)
- Mục tiêu rõ ràng: trả lời đúng, hữu ích, tự nhiên
- Chọn hành động hợp lý nhất (không cần toàn tri)

Chatbot hợp lý = hiểu input + tri thức + hành động tối ưu theo mục tiêu

3. Limited Rationality: Thảo luận

- **Câu hỏi 2: Ví dụ về Limited Rationality?**
- **Xe tự lái:** không tính hết mọi tình huống → chọn phương án an toàn, nhanh trong thời gian ngắn
- **Robot hút bụi:** không tối ưu đường đi toàn cục → dùng heuristic đơn giản để bao phủ nhanh
- **Chatbot/LLM:** không truy xuất toàn bộ kiến thức thế giới → trả lời dựa trên ngữ cảnh hiện có
- **Game AI:** tìm kiếm trong độ sâu giới hạn, không duyệt toàn bộ trạng thái

Limited rationality = hành động "đủ tốt, kịp thời", không cần tối ưu tuyệt đối

Vì sao môi trường quan trọng?

- Thiết kế agent phụ thuộc mạnh vào đặc trưng của môi trường.
- Các thuộc tính môi trường quyết định:
  - Kiến trúc agent
  - Chiến lược hành động
  - Hiệu năng có thể đạt được

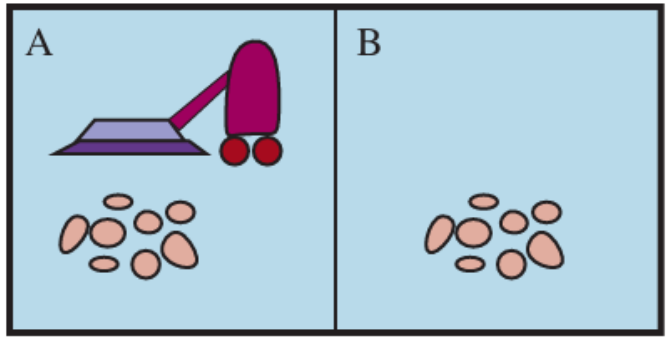


Figure 2.2 A vacuum-cleaner world with just two locations. Each location can be clean or dirty, and the agent can move left or right and can clean the square that it occupies. Different versions of the vacuum world allow for different rules about what the agent can perceive, whether its actions always succeed, and so on.

Phân loại môi trường -1

Fully observable vs. Partially observable

- Fully: sensors cung cấp toàn bộ thông tin cần để ra quyết định.
- Partially: thông tin không đầy đủ hoặc nhiễu.

Deterministic vs. Stochastic

- Deterministic: trạng thái tiếp theo hoàn toàn xác định.
- Stochastic: có yếu tố ngẫu nhiên.

Episodic vs. Sequential

- Episodic: mỗi quyết định độc lập (phân loại ảnh).
- Sequential: hành động ảnh hưởng về sau (chơi game).

Phân loại môi trường -2

Static vs. Dynamic

- Static: môi trường không thay đổi khi agent đang suy nghĩ.
- Dynamic: môi trường thay đổi theo thời gian.

Discrete vs. Continuous

- Discrete: hữu hạn trạng thái, hành động (cờ vua).
- Continuous: trạng thái/hành động liên tục (robot lái xe).

Single-agent vs. Multi-agent

- Single-agent: chỉ có một agent.
- Multi-agent: có nhiều agent, có thể cạnh tranh/hợp tác.

Bài tập nhanh: Phân loại môi trường

NETFLIX

1. **Robot dọn phòng (Roomba):** Partially observable, stochastic, sequential, dynamic, discrete, single-agent.
2. **Chơi cờ vua:** Fully observable, deterministic, sequential, static, discrete, multi-agent.
3. **Hệ thống gợi ý phim (Netflix recommender):** Partially observable, stochastic, sequential, dynamic, discrete, multi-agent (có nhiều user).

The Structure of Agents

- Agent = **architecture + program**
- Các loại chính:
  1. Simple reflex agent
  2. Model-based reflex agent
  3. Goal-based agent
  4. Utility-based agent
  5. Learning agent

Simple Reflex Agent

- Hoạt động dựa trên **rule "if condition → then action"**.
- Không có trí nhớ, chỉ dựa vào percept hiện tại.
- Ví dụ: robot hút bụi đơn giản "nếu thấy bụi → hút".

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

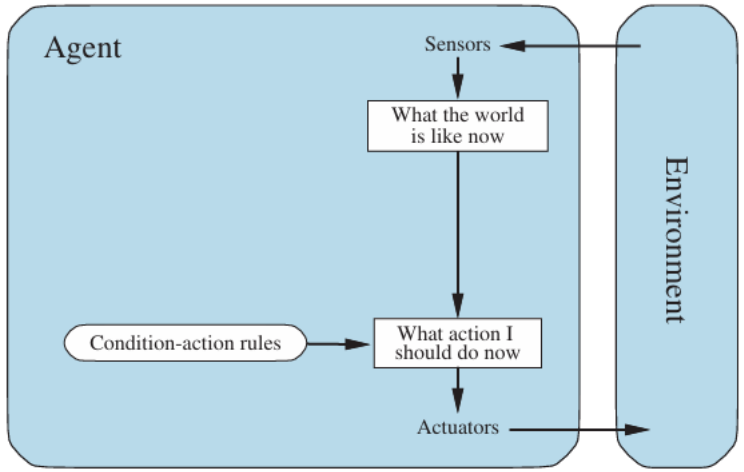


Figure 2.9 Schematic diagram of a simple reflex agent. We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

Model-based Reflex Agent

- Có **model (mô hình trạng thái)** lưu trữ thông tin quá khứ.
- Giúp xử lý môi trường **partially observable**.
- Ví dụ: chatbot duy trì ngữ cảnh hội thoại.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, a description of how the next state depends on
                    the current state and action
              sensor_model, a description of how the current world state is reflected
                    in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

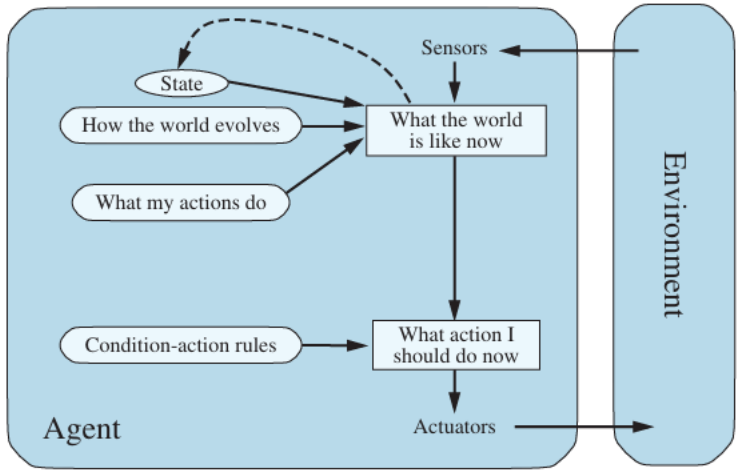


Figure 2.11 A model-based reflex agent.

Goal-based Agent

- Có **mục tiêu (goal)** để hướng hành động.
- Cần lập kế hoạch để đạt mục tiêu.
- Ví dụ: robot giao hàng chọn đường đến điểm đích.

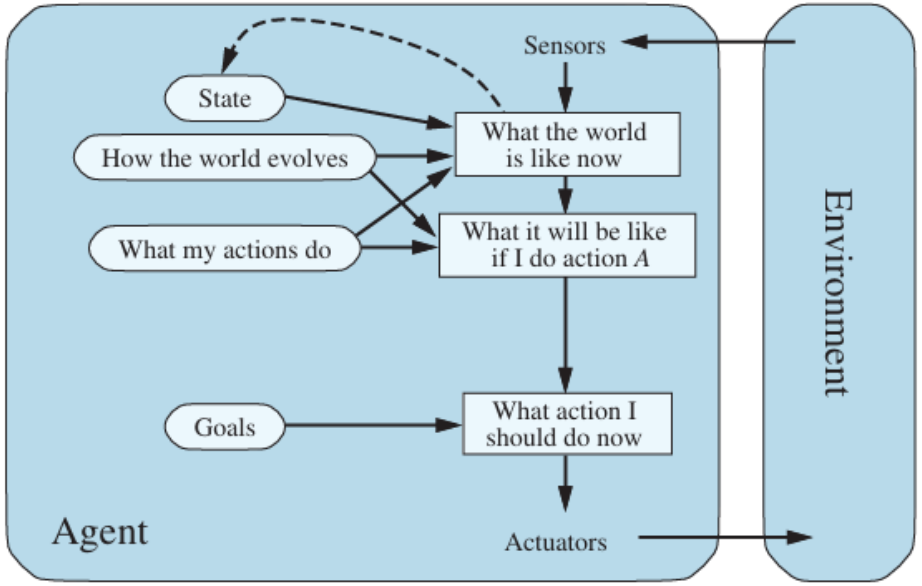


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Utility-based Agent

- Ngoài mục tiêu, có **utility function** đánh giá mức độ tốt/xấu của trạng thái.
- Giúp so sánh và chọn hành động tối ưu hơn.
- Ví dụ: ChatGPT chọn phản hồi “tự nhiên, mạch lạc, đúng ngữ cảnh” thay vì chỉ “đúng”.

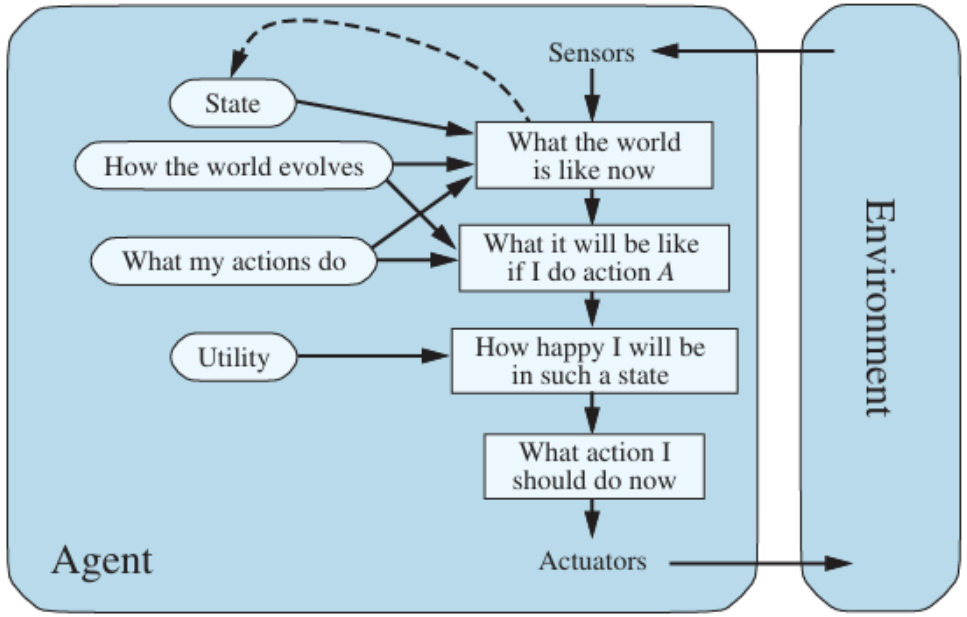


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning Agent

- Gồm 4 phần: Learning element, Performance element, Critic, Problem generator.
- Có khả năng **cải thiện hành vi theo thời gian**.
- Ví dụ: hệ thống chatbot học từ phản hồi người dùng.

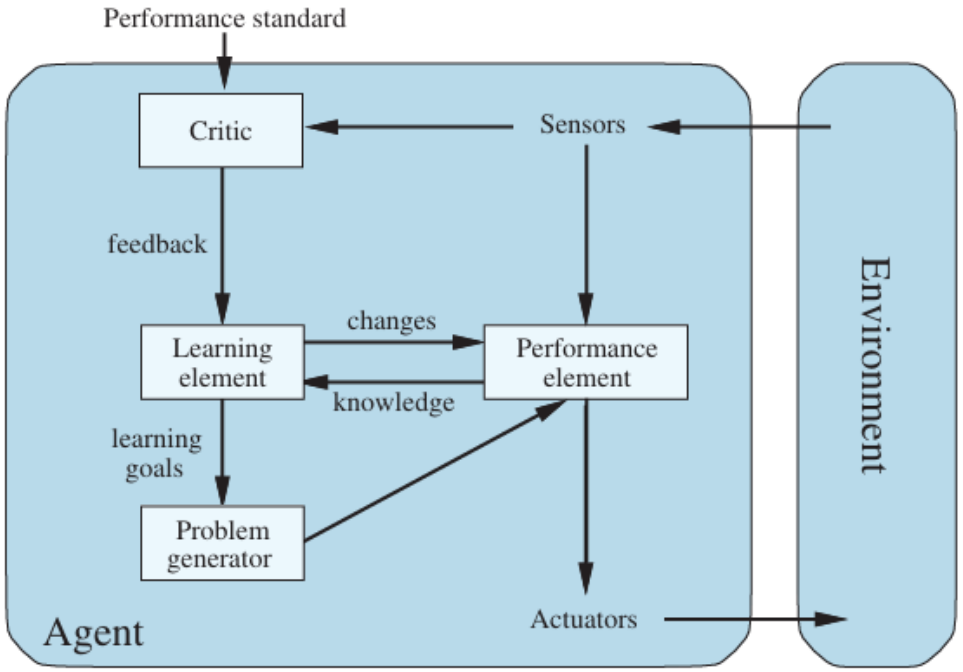


Figure 2.15 A general learning agent. The "performance element" box represents what we have previously considered to be the whole agent program. Now, the "learning element" box gets to modify that program to improve its performance.

Bài tập. Intelligent Agents

So sánh ba loại agent:

- Reflex agent (tác nhân phản xạ)
- Goal-based agent (tác nhân dựa trên mục tiêu)
- Utility-based agent (tác nhân dựa trên hàm hữu dụng)

Ba
Lu
lawrencium
103
Lr
rutherfordium
104
Rf
tantalum
73
Ta
dubnium
105
Db
tungsten
74
W
seaborgium
106
Sg
rhenium
75
Re
bohrium
107
Bh
Mo
43
Tc
Cr
Fe
Nd